

UNIT-V: Recursive and Recursively Enumerable Languages (11)

We have seen languages accepted by FA's are regular languages,
Languages accepted by PDA's are Context Free Languages.

→ Languages accepted by TM are Recursively Enumerable Languages

Recursively^{Enumerable} Languages (REL)

A Language L is said to be REL if L is the set of strings accepted by some TM.

i.e., If $w \in L$ then a TM halts in a final state.

If $w \notin L$ then a TM halts in a non-final state or loops forever.

→ REL's are also called as Turing Recognisable Languages

Recursive Language (RL)

A Recursive Language (subset of REL) can be decided by TM which means it will enter into final state for the strings of the language and rejection state for the strings which are not in the language.

i.e., If $w \in L$ then a TM halts in a final state

If $w \notin L$ then a TM halts in a non-final state.

→ RL is also called as Turing decidable Languages

Properties of RL

1. If L_1 and L_2 are two RL's, then their union $L_1 \cup L_2$ is also recursive language
2. If L_1 and L_2 are two RL's, then their intersection $L_1 \cap L_2$ is also RL.

3. The Complement of a RL ~~is~~ is also a RL.

4. The Kleene's closure of a RL is also a RL.

Universal Turing Machine (UTM)

A TM is said to be UTM if it can accept

(i) The input data

(ii) An algorithm (description) for computing

A UTM is just a TM whose programming simulates other TM's, i.e., the input to the UTM is a description of TM, T and an input for T , and the UTM simulates T on that input.

The Language

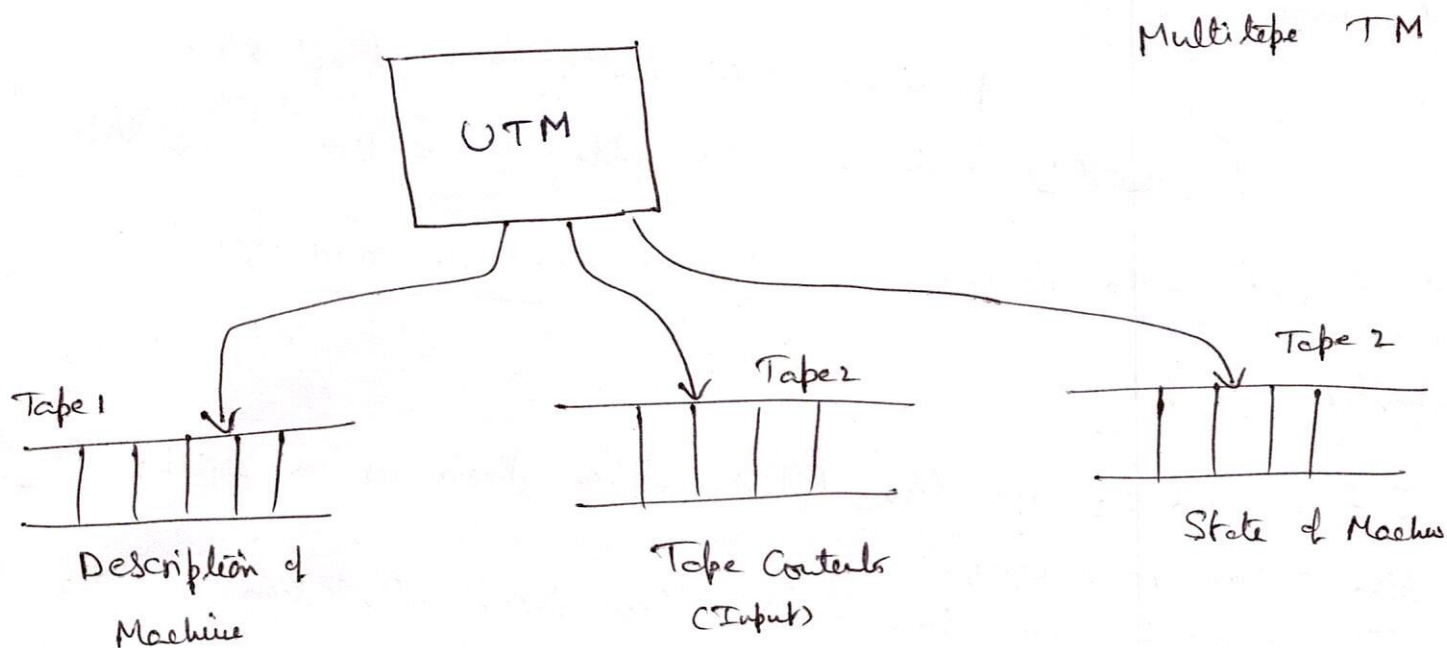
$$A_{TM} = \{ (M, w) \mid M \text{ is a TM and } M \text{ accepts } w \}$$

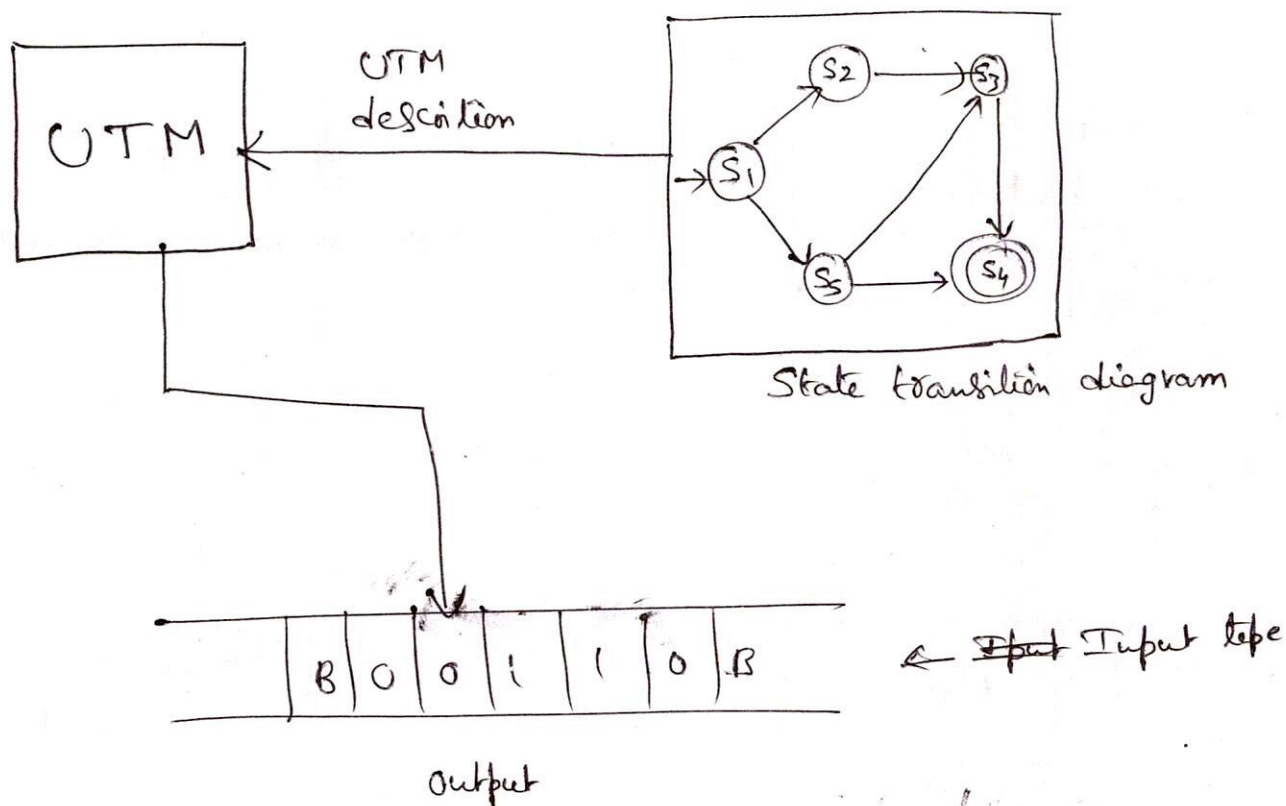
The UTM has three bits of information

(i) A basic description of Machine

(ii) The contents of the ~~tape~~ machine tape

(iii) The internal state of the machine





Here the State transition diagram is given as input i.e., it contains list of transition functions, it is called UTM description, it will be given to UTM as input.

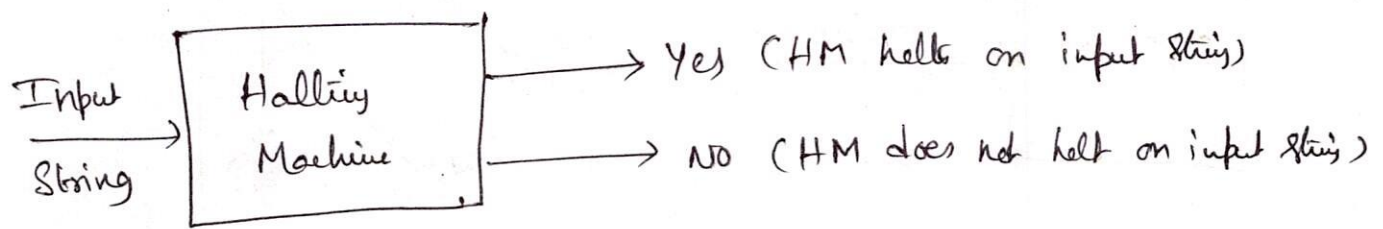
Then UTM simulates the ~~UTM~~ input from UTM description and the output is written on the input tape.

The Halting Problem

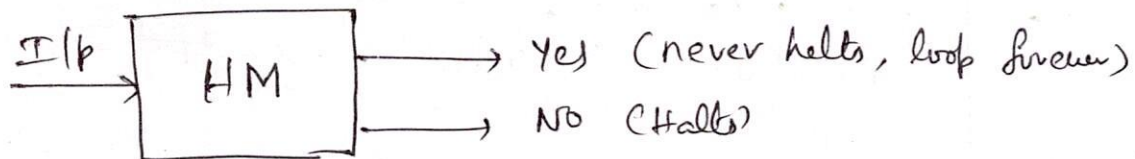
Halting means that the system will either accept & halt (reject) a certain input to avoid entering an infinite loop. The decision of whether & not a specific Turing machine should ~~halt~~ halt is known as the Halting Problem of TM's.

The TM acts a halting machine that produces a 'yes' & 'no' in a finite amount of time.

Block diagram of Halting Machine (HM)



Inverted halting Machine



The Halting Problem is called an ~~undecidable~~ undecidable problem as there is no algorithm which can predict whether a program will halt or not in a generalized way.

Undecidable Problems about TM's

Def: A problem is said to be decidable & solvable iff there exist an algorithm that takes as input an instance of the problem and determines whether the answer to that instance is Yes or No.

Def: A problem is said to be undecidable & unsolvable iff there is no algorithm that takes as input an instance of the problem and determines whether the answer to that instance is Yes or No.

→ A problem whose language is recursive is said to be decidable & solvable, otherwise the problem is undecidable & unsolvable.

For decidable problems solution is definite (Yes & no)

(3)

Ex: 1. Does sun rises in the East?

A) Yes

2. Does Earth moves around the sun?

A) Yes

For Undecidable problems solution is indefinite (i.e., sometimes Yes and sometimes No)

Ex: will tomorrow would be a rainy day?

A) We can't say (Undecidable)

There are three techniques to decide undecidability

1. Reducibility: Reduction is a technique in which if a problem P_1 is reduced to problem P_2 then any solution of P_2 solves P_1 . In general, if we have an algorithm to convert an instance of a problem P_1 to an instance of a problem P_2 that have the same answer then it is called as P_1 reduces to P_2 .

Hence if P_1 is not recursive then P_2 is also not recursive.

Similarly, if P_1 is not recursively enumerable then P_2 also is not recursively enumerable.

Theorem: If P_1 is reduced to P_2 , then

(i) If P_1 is undecidable, then P_2 is also undecidable

(ii) If P_1 is non-recursively enumerable, then P_2 is also non-RE.

2. Empty and Non-Empty languages

There are two types of languages Empty and non-Empty.

Let L_e denotes an Empty language and L_{ne} denotes a non-Empty language.

Let w be a binary string and M_i be a TM.

If $L(M_i) = \emptyset$, then M_i does not accept any input then w is in L_e .

If $L(M_i) \neq \emptyset$, then w is in L_{ne} . Thus we can say that

$$L_e = \{M \mid L(M) = \emptyset\}$$

$L_{ne} - \text{RE}$

$$L_{ne} = \{M \mid L(M) \neq \emptyset\}$$

$L_e - \text{not RE}$

Both L_e and L_{ne} are Complement of one another.

3. Rice's Theorem:

Every non-trivial (answer is not known) problem on RE L's is undecidable.

Ex: If a language is RE, its Complement will be RE or not is undecidable.

Context Sensitive Language (CSL)

Context Sensitive grammar (CSG) has 4-tuples

$$G = (V, T, P, S)$$

when P are of the form $\alpha_1 A \alpha_2 \rightarrow \alpha_1 \beta \alpha_2$

when $\alpha_1, \alpha_2 \in (V \cup T)^*$, $A \in V$, $\beta \in (V \cup T)^+$

and a rule of the form $S \rightarrow \epsilon$ is allowed if the start symbol S do not appear on R.H.S of any rule.

→ The language generated by CSG is called a CSL

→ Every CFG is also a CSG. The CFL are subset of CSL.

(4)

Ex: $L = \{a^n b^n c^n \mid n \geq 1\}$ is a CSL but not CFL

The CSG for this L is

$$S \rightarrow aSBC / aBC$$

$$CB \rightarrow HB,$$

$$HB \rightarrow HC,$$

$$HC \rightarrow BC,$$

$$aB \rightarrow ab,$$

$$bB \rightarrow bb,$$

$$bC \rightarrow bc,$$

$$cC \rightarrow cc$$

To derive the string $aabbcc$,

$$S \rightarrow a \underline{S} BC$$

$$\rightarrow a \underline{a} BC BC \quad (\because S \rightarrow aBC)$$

$$\rightarrow a ab \underline{C} BC \quad (\because aB \rightarrow ab)$$

$$\rightarrow a ab \underline{H} BC \quad (\because CB \rightarrow HB)$$

$$\rightarrow a ab \underline{H} CC \quad (\because HB \rightarrow HC)$$

$$\rightarrow a a \underline{b} BCC \quad (\because HC \rightarrow BC)$$

$$\rightarrow a a b \underline{b} CC \quad (\because bB \rightarrow bb)$$

$$\rightarrow a a b b \underline{c} C \quad (\because bC \rightarrow bc)$$

$$\rightarrow a a b b c c \quad (\because cC \rightarrow cc)$$

Linear Bounded Automata (LBA)

A LBA is a nondeterministic Turing Machine which has a single tape whose length is not infinite but bounded by a linear function of the length of the input string.

It can be defined by a 9-tuple $(Q, \Sigma, \Gamma, \delta, q_0, b, \phi, \$, F)$

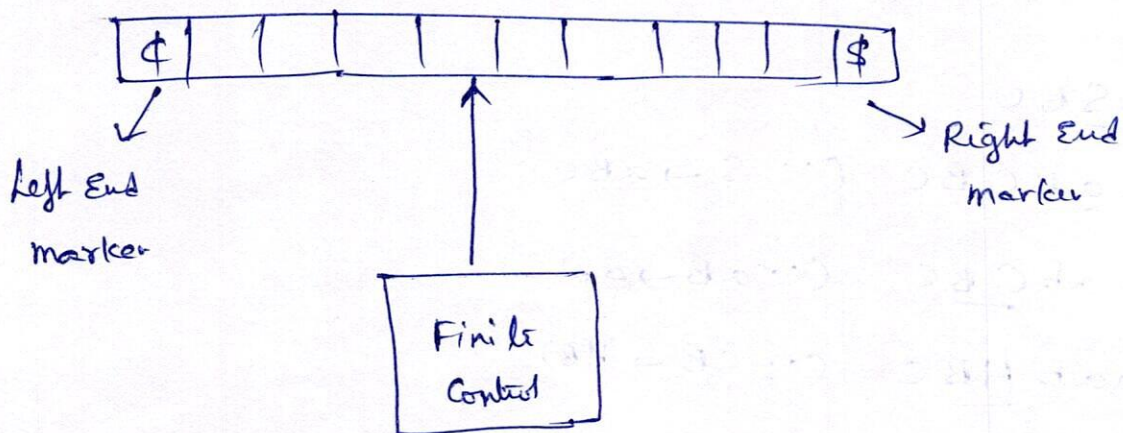
here all the symbols have same meaning as in the TM

except $\phi, \$$

ϕ is left-end marker

$\$$ is right-end marker $\phi, \$ \in \Sigma$

If L is a CSL,
then L is accepted by
a LBA.



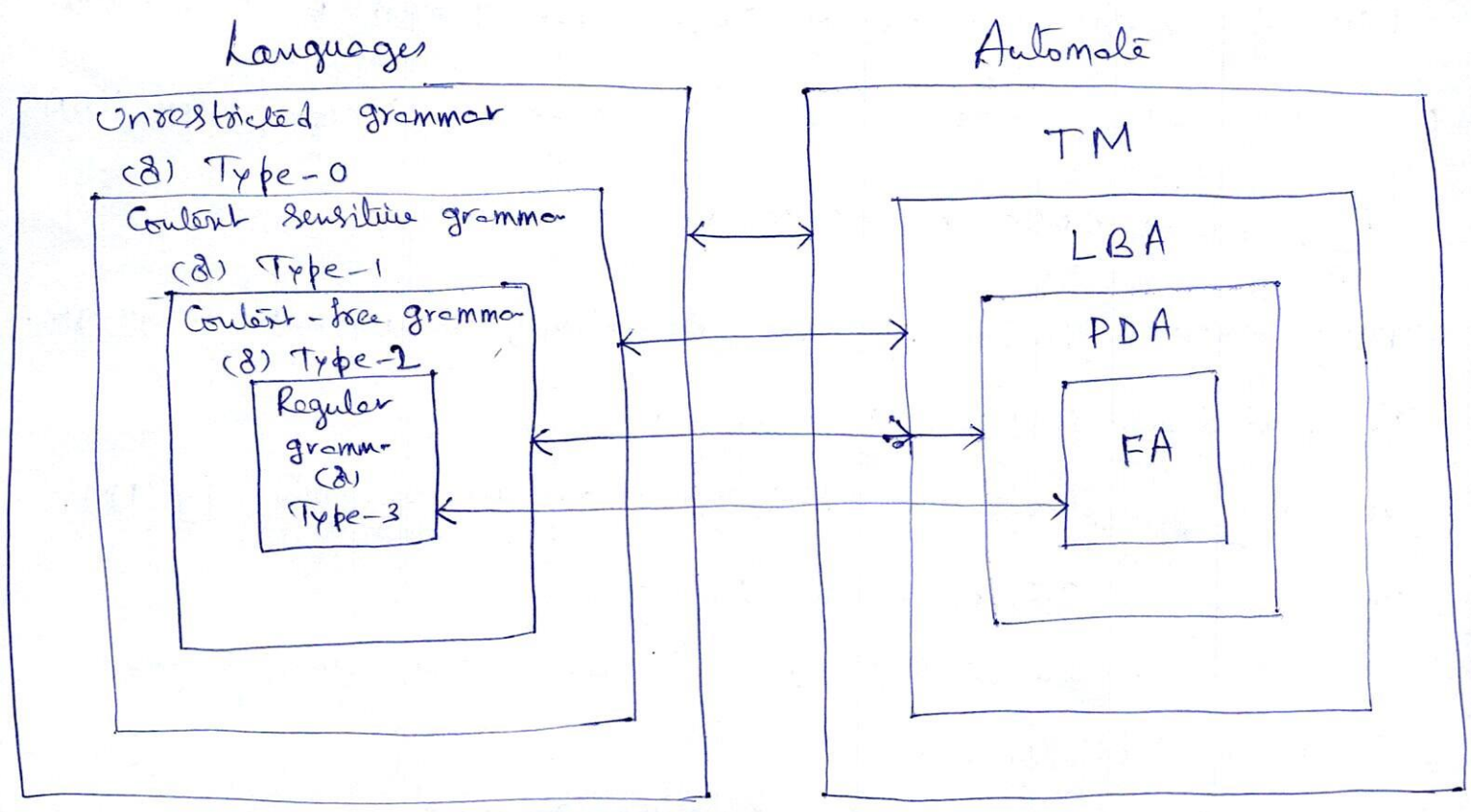
Chomsky Hierarchy

According to Noam Chomsky, there are four types of grammars

Type 0, Type 1, Type 2, Type 3.

Grammar Type	Grammar Accepted	Language Accepted	Automaton
Type-0	Unrestricted grammar	Recursively Enumerable language	Turing Machine
Type-1	Context sensitive grammar	Context sensitive language	Linear Bounded Automaton
Type-2	Context free grammar	Context free language	Push down Automaton
Type-3	Regular Grammar	Regular language	Finite Automaton

Type-3 Grammar : It generates regular languages.



Type-3 Grammar: It generates Regular languages.

The productions must be in the form of

$$V \rightarrow T^* V \mid T^*$$

V - non terminal

T - terminal

Ex: $A \rightarrow aB$, $A \rightarrow a$, $A \rightarrow \epsilon$, $A, B \in V$, $a \in T$

Type-2 Grammar: It generates Context free languages.

The productions must be in the form

$$A \rightarrow \alpha, \text{ where } A \in V \text{ and } \alpha \in (V \cup T)^*$$

The language generated by these grammars are recognised by PDA

Ex: $A \rightarrow aBb$, $A \rightarrow b$, $B \rightarrow a$

Type-1 Grammar: It generates Context Sensitive languages.

The productions must be in the form

$$\alpha A \beta \rightarrow \alpha \gamma \beta, \text{ where } A \in V, \alpha, \beta, \gamma \in (V \cup T)^*$$

Here the strings α, β may be empty, but γ must be non-empty.

The languages generated by these grammars are recognised by LBA

Ex: $AB \rightarrow AbBc$, $A \rightarrow bCA$, $B \rightarrow b$

Type-0 Grammar: It generates Recursively Enumerable languages.

The productions have no restrictions.

The languages generated by these grammars are recognised by TM.

The productions can be in the form of

$$\alpha \rightarrow \beta, \text{ where } \alpha, \beta \in (V \cup T)^* \text{ and } \alpha \neq \epsilon \text{ and } \alpha \text{ having at least one non-terminal}$$

Ex: $S \rightarrow AcaB$, $Bc \rightarrow acB$, $CB \rightarrow \lambda B$, $aB \rightarrow \lambda B$

Post's Correspondence Problem (PCP)

⑥

An instance of PCP consists of two lists of strings over some alphabet Σ . The two lists must be of equal length.

The two lists are $A = w_1, w_2, \dots, w_n$ and $B = x_1, x_2, \dots, x_n$

then there exists a non-empty set of integers $i_1, i_2, \dots, i_k$ such that

$$w_{i_1} w_{i_2} \dots w_{i_k} = x_{i_1} x_{i_2} \dots x_{i_k}$$

we say the sequence $i_1, i_2, \dots, i_k$ is a solution to this instance of PCP, if so. (i.e., the lists A and B yield the same string)

① Find whether the lists over $\Sigma = \{0, 1\}$

$A = (1, 10111, 10), B = (111, 10, 0)$ have a Post Correspondence solution?

Sol:

	A	B
1	$1 = w_1$	$111 = x_1$
2	$10111 = w_2$	$10 = x_2$
3	$10 = w_3$	$0 = x_3$

Step ①: First, we will start with a pair (w_i, x_i) in which the first symbol in w_i and the first symbol in x_i is same. Here the pairs (w_1, x_1) and (w_2, x_2) have the same symbols in first position.

A : 1

B : 111

A : 10111
B : 10

Step ②: For the pair (w_1, x_1) we need 1's in the numerator to match the 1's in the denominator. So we have to use only the (w_1, x_1) again and again, But we can't get same string in A and B.

Hence leave this first cell.

Now for the pair (w_2, x_2) , we need 1's in the denominator to match the 1's in the numerator. So we will go with the pair (w_1, x_1)

A: 101111

B: 10111

Step 3: Now, there is an entire '1' in the numerator and to match this 1, we will use the pair (w_1, x_1) again

A: 10111111

B: 10111111

Step 4: Now, there is an entire '1' in the denominator, and to match it, we will add the third pair (w_3, x_3) .

A: 101111110

B: 101111110

Hence the sequence of selection is:

$$w_2 w_1 w_1 w_3 = x_2 x_1 x_1 x_3$$

$$\text{i.e., } 2113. \text{ i.e., } i_1 = 2, i_2 = 1, i_3 = 1, i_4 = 3$$

Note: Here the solution is not unique.

② Does PCP with two lists $X = (b, a, aba, bb)$, $Y = (ba, ba, ab, b)$ have a solution?

Sol

	x	y
1	b	ba
2	a	ba
3	aba	ab
4	bb	b

$x: b$ ① $x: aba$ ③ $x: bb$ ④
 $y: ba$ $y: ab$ $y: b$

$x: ba$ ①② $x: abaaba$ X $x: bba$ ④② but it contains ① & ③ also
 $y: baba$ $y: abab$ $y: bba$

$x: bab$ ①②①
 $y: bababa$

$x: bababa$ ①②①③
 $y: bababab$

$x: babababab$ ①②①③③
 $y: babababab$

$x: bababababbb$ ①②①③③④
 $y: bababababbb$

$$\therefore w_1 w_2 w_1 w_3 w_3 w_4 = x_1 x_2 x_1 x_3 x_3 x_4$$

③ Does PCP with two lists

$A = (10, 011, 101)$, $B = (101, 11, 011)$ have a solution?

Sol:

	A	B
i	w_i	x_i
1	10	101
2	011	11
3	101	011

A : 10

①

B : 101

A : 10 1 0 ① ① X

B : 10 1 1 0 1

A : 10 0 1 1 ① ② X

B : 10 1 1 1

A : 10 10 1 ① ③

B : 10 10 1 1

A : 10 10 1 1 0 1 ① ③ ③

B : 10 10 1 1 0 1 1 ...

This loop will continue
infinitely

Hence this PCP has no solution